

Databases & SQL

What Databases Are, Their Types, and How SQL Works

A database is one of the most fundamental building blocks of modern software — almost every app, website, and business system relies on one to store and retrieve information. This guide explains what a database is, the major types of databases in use today, how a Database Management System (DBMS) works, and how SQL, the standard language for relational databases, is used to create, query, and manage data.

1. What Is a Database?

A database is an organized collection of structured (or semi-structured) data, stored electronically so it can be easily accessed, managed, and updated. Rather than storing information in loose files, a database organizes data in a consistent structure — such as tables, documents, or key-value pairs — that software can query efficiently.

A **Database Management System (DBMS)** is the software that actually creates, reads, updates, and deletes data within a database, while also handling tasks like security, backups, and multi-user access control.

2. Types of Databases

2.1 Relational Databases (SQL)

Store data in **tables** made up of rows and columns, where relationships between tables are defined using keys. Relational databases are queried using SQL (Structured Query Language) and are known for strong consistency and structured data integrity.

Examples: MySQL, PostgreSQL, Microsoft SQL Server, Oracle Database, SQLite.

2.2 NoSQL Databases

A broad category of databases that don't use the traditional table-based relational model, designed for flexibility, scalability, and handling unstructured or rapidly changing data. NoSQL databases break down into several sub-types:

- **Document databases** — store data as flexible, JSON-like documents (e.g., MongoDB, Couchbase)
- **Key-value databases** — store simple pairs of unique keys and values, optimized for speed (e.g., Redis, Amazon DynamoDB)
- **Column-family databases** — store data in columns rather than rows, ideal for large analytical datasets (e.g., Apache Cassandra, HBase)
- **Graph databases** — store data as nodes and relationships, ideal for highly connected data like social networks (e.g., Neo4j, Amazon Neptune)

2.3 Hierarchical Databases

Organizes data in a tree-like structure, where each record has a single parent but can have multiple children. Common in legacy systems and certain specialized applications like file systems and organizational charts.

Examples: IBM Information Management System (IMS), Windows Registry (conceptually similar structure).

2.4 Network Databases

An extension of the hierarchical model where a record can have multiple parent and child records, forming a more flexible, graph-like network of connections. Largely a historical model, mostly superseded by relational and graph databases.

Examples: Integrated Data Store (IDS), IDMS (historical systems).

2.5 Object-Oriented Databases

Stores data in the form of objects, similar to how data is represented in object-oriented programming languages, allowing complex data types to be stored directly without translation.

Examples: db4o, ObjectDB.

2.6 Cloud Databases

Databases built to run on cloud infrastructure, offering automatic scaling, managed maintenance, and access from anywhere. Can be relational or NoSQL under the hood.

Examples: Amazon RDS, Google Cloud SQL, Microsoft Azure SQL Database, Amazon Aurora.

2.7 Data Warehouses

Specialized databases designed for large-scale analysis and reporting rather than everyday transactions, typically consolidating data from multiple sources for business intelligence.

Examples: Snowflake, Google BigQuery, Amazon Redshift.

3. Quick Reference Table

Type	Data Structure	Best For	Examples
Relational (SQL)	Tables (rows/columns)	Structured data, transactions	MySQL, PostgreSQL
Document (NoSQL)	JSON-like documents	Flexible, evolving schemas	MongoDB
Key-value (NoSQL)	Key/value pairs	Caching, high-speed lookups	Redis
Column-family	Column-oriented	Big data, analytics	Cassandra
Graph	Nodes and edges	Connected/relationship data	Neo4j
Data Warehouse	Large structured datasets	Business intelligence, reporting	Snowflake

4. Key Concepts in Relational Databases

4.1 Tables, Rows, and Columns

Data is organized into **tables**, where each **row** (or "record") represents a single item, and each **column** (or "field") represents an attribute of that item.

4.2 Primary Keys

A **primary key** is a column (or combination of columns) that uniquely identifies each row in a table, ensuring no two rows are identical and enabling fast lookups.

4.3 Foreign Keys

A **foreign key** is a column in one table that references the primary key of another table, creating a link (relationship) between the two tables.

4.4 Normalization

The process of organizing tables to reduce data redundancy and improve data integrity, typically by splitting data into multiple related tables rather than repeating it in one large table. Often described in stages called "normal forms" (1NF, 2NF, 3NF).

5. Introduction to SQL

SQL (Structured Query Language) is the standard language used to communicate with relational databases — to create structures, insert and modify data, and retrieve information through queries.

5.1 Creating a Table

```
CREATE TABLE Employees (  
  EmployeeID INT PRIMARY KEY,  
  Name VARCHAR(100),  
  Department VARCHAR(50),  
  Salary DECIMAL(10,2)  
);
```

5.2 Inserting Data

```
INSERT INTO Employees (EmployeeID, Name, Department, Salary)  
VALUES (1, 'Ama Boateng', 'Marketing', 3500.00);
```

5.3 Querying Data (SELECT)

```
SELECT Name, Salary FROM Employees WHERE Department = 'Marketing';
```

Key clauses used with SELECT:

- **WHERE** — filters rows based on a condition
- **ORDER BY** — sorts results (ASC for ascending, DESC for descending)
- **GROUP BY** — groups rows sharing a value, often used with aggregate functions
- **HAVING** — filters groups created by GROUP BY (similar to WHERE, but for grouped data)
- **LIMIT** (or TOP in SQL Server) — restricts the number of rows returned

5.4 Updating Data

```
UPDATE Employees SET Salary = 3800.00 WHERE EmployeeID = 1;
```

5.5 Deleting Data

```
DELETE FROM Employees WHERE EmployeeID = 1;
```

6. Combining Tables with JOIN

Joins let you combine rows from two or more tables based on a related column, usually a foreign key relationship.

6.1 INNER JOIN

Returns only rows that have matching values in both tables.

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
INNER JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

6.2 LEFT JOIN

Returns all rows from the left table, and matching rows from the right table (NULL if no match).

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
LEFT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

6.3 RIGHT JOIN and FULL JOIN

A **RIGHT JOIN** mirrors a LEFT JOIN but keeps all rows from the right table instead. A **FULL JOIN** (or FULL OUTER JOIN) returns all rows from both tables, matching where possible.

7. Aggregate Functions

Function	Purpose	Example
COUNT()	Counts rows	SELECT COUNT(*) FROM Employees;
SUM()	Adds numeric values	SELECT SUM(Salary) FROM Employees;
AVG()	Calculates the average	SELECT AVG(Salary) FROM Employees;
MIN() / MAX()	Finds smallest/largest value	SELECT MAX(Salary) FROM Employees;

8. Data Integrity: ACID Properties

Relational databases aim to guarantee reliable transactions through four properties known as **ACID**:

- **Atomicity** — a transaction either completes fully or not at all
 - **Consistency** — a transaction brings the database from one valid state to another
 - **Isolation** — concurrent transactions don't interfere with each other
 - **Durability** — once a transaction is committed, it remains saved even after a system failure
-

9. Advantages & Disadvantages

9.1 Relational (SQL) Databases

- **Advantages:** Strong data integrity, mature tooling, powerful and standardized query language, well-suited to structured/transactional data
- **Disadvantages:** Rigid schema (harder to change structure later), can be harder to scale horizontally across many servers

9.2 NoSQL Databases

- **Advantages:** Flexible/schema-less design, easier horizontal scaling, well-suited to large volumes of unstructured or rapidly changing data
 - **Disadvantages:** Often weaker consistency guarantees, less standardized query languages across products, can require more careful data modeling upfront
-

10. Summary

A database organizes data so it can be reliably stored, retrieved, and updated, with a DBMS handling the underlying management. Relational databases use structured tables and the SQL language, offering strong consistency and a mature ecosystem, while NoSQL databases (document, key-value, column-family, and graph) trade some of that structure for flexibility and easier scaling. Understanding core relational concepts — keys, normalization, joins, and SQL queries — provides a strong foundation for working with almost any modern data-driven application.